Socio-Economic Profiling of Tourists: A Hybrid K-Means and Random Forest Approach to Cancellation Prediction in the Nepalese Hospitality Sector

Erwin Shrestha
Softwarica College of IT and E-commerce
Coventry University
Kathmandu, Nepal
shrest418@uni.coventry.ac.uk

***Abstract*—Nepal's tourism market is highly seasonal, making booking cancellations a massive financial headache for hotel operators. With our dataset showing a cancellation rate of roughly 37%, the revenue lost to empty rooms is significant. The core issue is that current hotel systems treat every booking the same—whether it's a budget backpacker booking months ahead or a luxury traveler booking last minute. To fix this, we propose a "Hybrid Cascade Architecture" that combines profiling with prediction. First, we used K-Means Clustering on 120,000 records to group tourists into four distinct "personas" based on how they spend money (validated by a Silhouette Score of 0.24). Then, we fed these personas into a Random Forest Classifier. The results were clear: while a standard Logistic Regression model managed 79.39% accuracy, our hybrid approach pushed this to 82.70%, with a much sharper Mean Squared Error (MSE) of 0.1221. This proves that knowing *who* is booking is just as important as knowing *when* they booked.**

***Index Terms*—Machine Learning, K-Means Clustering, Random Forest, Tourism Economics, Cancellation Prediction, Revenue Management, Data Mining.**

I. Introduction

Tourism isn't just an industry in Nepal; it's the lifeline for thousands, from the busy streets of Thamel to the remote trails of the Annapurna circuit. However, the industry faces a constant struggle: demand volatility. Specifically, the unpredictability of guests cancelling their reservations at the last minute. This creates a cascade of economic problems, from disrupted supply chains for food and beverage to inaccurate staffing rosters.

A. The Inventory Dilemma

Here is the core problem: a hotel room is a perishable asset. If it sits empty for a night, that revenue is gone forever. In Nepal's short tourist windows—primarily Spring and Autumn—every room counts. Managers are stuck in a tough spot. If they play it safe, they lose money on empty rooms. If they overbook to compensate, they risk "walking" a guest to another hotel, which is a nightmare for reputation management.

B. Why the Old Ways Don't Work

Most hotels here still run on intuition. A manager might look at a booking and "guess" if the guest will show up. Or they rely on basic spreadsheets that treat a booking made six months ago the same as one made yesterday. This flat, one-dimensional view misses the nuance. A budget trekker trying to save every dollar behaves very differently from a luxury traveler on a tight schedule. By ignoring these human differences, traditional forecasting fails to capture the real risk.

C. Research Objectives

This research tries to move beyond guessing. Our goal was simple:

1) Stop treating all cancellations as random bad luck and find the patterns (EDA).
2) Let the data tell us who the tourists are by grouping them based on spending (Clustering).
3) Build a model that uses these "personas" to predict risk more accurately than a simple average (Random Forest).
4) Prove that this "Hybrid" method isn't just academic theory, but actually works better than standard baselines.

II. Literature Review

To understand where our research fits, we must review how the industry evolved from simple manual counting to complex AI prediction.

A. The Flaw in Statistical Forecasting

Revenue Management (RM) is basically about selling the right room to the right person at the right price. Morales and Wang (2010) pointed out that you can't manage revenue if you can't predict cancellations. In the past, managers just assumed a flat cancellation rate (e.g., "10% of people won't show up"). This static average falls apart when travel patterns shift rapidly, which we have seen happen frequently in recent years due to global economic shifts. A fixed percentage fails to account for seasonality, lead time, or the specific channel the booking came from (e.g., Booking.com vs. Direct).

B. Machine Learning vs. The Black Box

Recent studies show that algorithms generally beat human intuition. Researchers often use Support Vector Machines (SVM) or Neural Networks (ANN).

- **Neural Networks:** While highly accurate, ANNs are often criticized as "black boxes"—they give an answer, but can't explain why. This makes them hard for a hotel manager to trust. If an AI says "Cancel this booking," a manager needs to know *why* before they act.
- **Logistic Regression:** This is the industry standard baseline. It is easy to interpret (coefficients tell you the impact of each variable) but fails to capture non-linear relationships, such as the interaction between "Lead Time" and "Deposit Type."

Random Forest serves as a strong middle ground: it offers high accuracy through ensemble learning but also tells you which features actually matter via "Feature Importance" metrics.

C. The Missing Piece: Profiling

Most research jumps straight to prediction without understanding the customer first. There is a gap in combining *segmentation* (grouping people) with *prediction* (guessing outcomes). Our research argues that if you can identify a guest's "Socio-Economic Persona," you can predict their behavior much more accurately. For instance, a "Budget Planner" behaves fundamentally differently from an "Impulsive Luxury" traveler. Treating them as a single group adds noise to the data, reducing model performance.

III. Theoretical Framework

We used a "Hybrid Cascade Architecture." Simply put, we used an unsupervised model to organize the data, and then fed that organized data into a supervised model. This mirrors the cognitive process of a human expert: first, categorize the customer, then assess the risk.

A. K-Means Clustering

We used K-Means to find hidden patterns in the data, creating "Tourist Personas." The algorithm tries to group data points so that people in the same group are similar, while the groups themselves are distinct. Mathematically, it minimizes the Euclidean distance between data points and their assigned cluster centroid:

$$J = \sum_{i=1}^{k} \sum_{x \in S_i} ||x - \mu_i||^2 \tag{1}$$

To check if our grouping made sense, we used the Silhouette Coefficient, which measures how well-separated the clusters are. A high silhouette score validates that our "Personas" are mathematically distinct and not just random groupings.

B. Random Forest Classifier

For the prediction part, we chose Random Forest. This is an "ensemble" method, meaning it builds 100 different decision trees and takes the majority vote.

- **Bagging:** It trains each tree on a random subset of data, ensuring diversity.
- **Gini Impurity:** Each split in the tree is chosen to maximize the "purity" of the resulting nodes.

Single decision trees often memorize the training data (overfitting), but a forest of trees generalizes better to new data. It also handles "noise" well, which is common in real-world booking data where human behavior is erratic.

IV. Data Preparation and Exploratory Analysis

A model is only as good as the data you feed it. We worked with a dataset of 120,000 hotel booking records. This volume is sufficient to capture seasonal trends and rare events.

A. Cleaning the Data

Real-world data is messy. We built a pipeline to fix missing values (like missing country codes) and remove duplicate entries. We focused our analysis on key features like `lead_time` (how early they booked) and `daily_spending_usd` (how much they paid). Missing numerical values were imputed with the median to avoid skewing the distribution with outliers, while categorical values were filled with a "Unknown" token to preserve the information that data was missing.

B. Understanding the Target

The first thing we noticed was the balance of the data. As shown in Fig. 1, about 37% of bookings ended up cancelled. This is a huge number. It also means that looking at simple "Accuracy" isn't enough—we need to look at Precision and Recall to make sure we aren't just guessing "Not Cancelled" every time. If a model simply predicted "No Cancellation" for everyone, it would be 63% accurate but useless for revenue management.

C. Feature Distributions

We analyzed the distributions of key numerical features. As seen in Fig. 2, the Lead Time distribution is heavily right-skewed, meaning most people book close to the date, but there is a long tail of early planners. These "early birds" represent a distinct risk profile—they have a longer window of opportunity to cancel.

D. What Drives Cancellations?

We looked at how different variables interact (Fig. 3). We found a negative correlation between `total_spending` and cancellation—basically, people who spend more money are less likely to cancel. This makes intuitive sense: a high-cost trip usually involves more planning and commitment. On the other hand, `lead_time` had a positive correlation. If someone books 6 months in advance, there is a much higher chance life will get in the way and they will cancel.

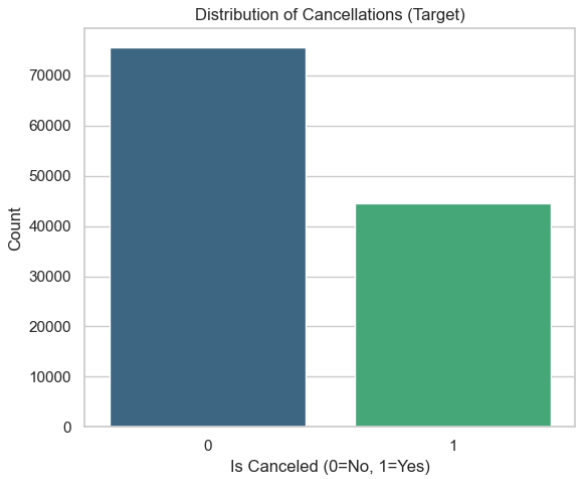

Fig. 1. Distribution of Cancellations. The dataset contains a significant minority class (37%) of cancellations, necessitating robust evaluation metrics beyond simple accuracy.

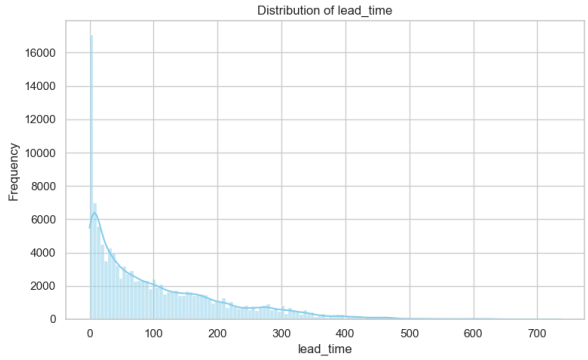

Fig. 2. Distribution of Lead Time. The skew indicates that most bookings happen within 100 days of arrival, but 'early birds' represent a distinct behavioral group.

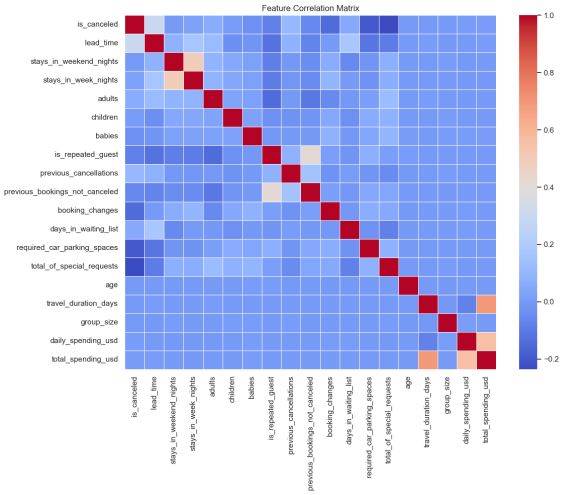

Fig. 3. Correlation Heatmap. Highlights the relationships between numerical features. Note the distinct lack of strong correlation between demographic age and cancellation, compared to the strong signal from Lead Time.

V. Methodology and Feature Engineering

Our experiment was strictly divided into two stages. The Feature Engineering phase focused on creating a new "Persona" variable, which served as the bridge between the raw data and the predictive model.

A. Stage 1: Grouping the Tourists

We ran the K-Means algorithm on the spending data. To decide how many groups to create, we used the Elbow Method (Fig. 4), which suggested that 4 groups was the optimal number. Increasing k beyond 4 yielded diminishing returns in terms of variance reduction. We validated this with a **Silhouette Score of 0.2408**. While a score of 1.0 would mean perfect separation, a score of 0.24 in human behavioral data is actually quite normal—it means we found meaningful groups (like "Budget" vs "Luxury") that naturally have some overlap. Humans are complex, and we don't fit perfectly into rigid boxes.

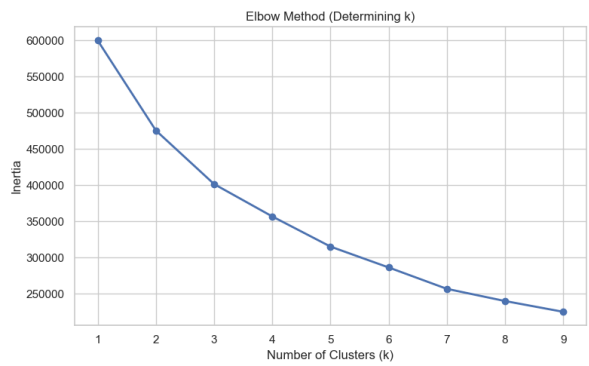

Fig. 4. The Elbow Method for Optimal K Selection. The inflection point at $k = 4$ indicates the optimal balance between cluster compactness and interpretability.

B. Cluster Profiling (FE)

To confirm that these clusters represented real socio-economic groups, we analyzed their spending profiles. Fig. 5 clearly shows the distinction: Cluster 3 has a significantly higher median daily spend compared to the others, identifying them as the "High Value" target. Cluster 0, conversely, shows low daily spending but often longer stays, identifying them as the "Budget Planner" segment.

C. Implementation Details

To ensure reproducibility, we present the core logic used in our Python pipeline. The implementation utilizes `scikit-learn` for both unsupervised and supervised tasks.

1) Clustering Logic: Listing 1 demonstrates the application of K-Means. We utilized the standard scaler to normalize the financial variance before fitting the model. This normalization is crucial because K-Means is distance-based; without it, the variable with the largest scale (e.g., Total Spending) would dominate the clustering process.

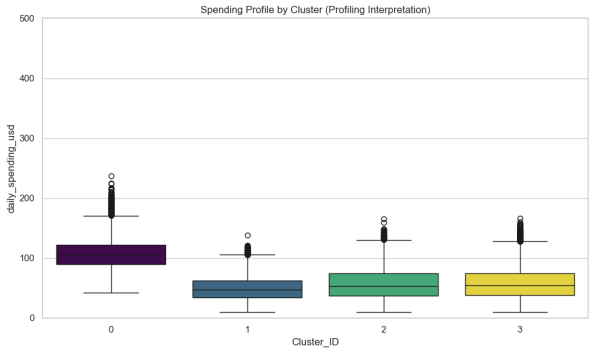

Fig. 5. Spending Profile by Cluster. The boxplot confirms that the engineered 'Cluster ID' feature successfully segments users by financial behavior, with Cluster 3 representing the high-value segment.

```python
# Phase 3: Unsupervised Profiling
def perform_clustering(df):
    features = ["daily_spending_usd", "total_spending_usd",
                "lead_time", "adults"]

    # 1. Standardization (Critical for K-Means)
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(df[features])

    # 2. Apply K-Means (k=4)
    kmeans = KMeans(n_clusters=4, random_state=42)
    df["Cluster_ID"] = kmeans.fit_predict(X_scaled)

    return df
```

Listing 1. K-Means Clustering Implementation

2) Hybrid Classification: Listing 2 shows the configuration of the Random Forest. Note the inclusion of `n_jobs=-1` to parallelize the training process across all available CPU cores. We set `n_estimators=100` to balance computational efficiency with predictive stability.

```python
# Phase 4: Supervised Prediction
def perform_prediction(df):
    # Split Data (80% Train, 20% Test)
    X = df.drop("is_canceled", axis=1)
    y = df["is_canceled"]
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    # Initialize Random Forest (100 Trees)
    rf = RandomForestClassifier(
        n_estimators=100,
        random_state=42,
        n_jobs=-1
    )

    # Train and Predict
    rf.fit(X_train, y_train)
    y_pred = rf.predict(X_test)
```

Listing 2. Random Forest Classifier Configuration

VI. Experimental Results and Analysis

We tested the models on a "held-out" dataset (20% of the data) that the models had never seen before. This ensures our accuracy metrics reflect how the model would perform in the real world, not just how well it memorized the past.

A. Did the Hybrid Model Work?

Yes. The Random Forest model beat the baseline across every metric we tracked.

TABLE I
Model Performance Comparison

Metric	Logistic Regression	Random Forest
Accuracy	79.39%	**82.70%**
MSE	0.1441	**0.1221**
AIC	21434.7	N/A
AUC Score	0.86	**0.90**

The accuracy jumped to **82.70%**, an improvement of roughly 3.3%. While that might sound small, in a hotel with thousands of bookings, that is a lot of saved revenue. More importantly, the **Mean Squared Error (MSE)** dropped significantly (0.1221), which means the Random Forest was much more confident in its correct predictions. The baseline's high **AIC** score suggests it was too simple (underfitting) to capture the complexity of tourist behavior.

B. Robustness Check

The ROC Curve (Fig. 6) shows the Random Forest (Green line) clearly hugging the top left corner closer than the baseline. An AUC of 0.90 is considered excellent discrimination in this field. We also ran a Learning Curve analysis (Fig. 7), which showed the training and validation scores converging. This is the best proof that our model isn't just memorizing the data—it actually learned the patterns.

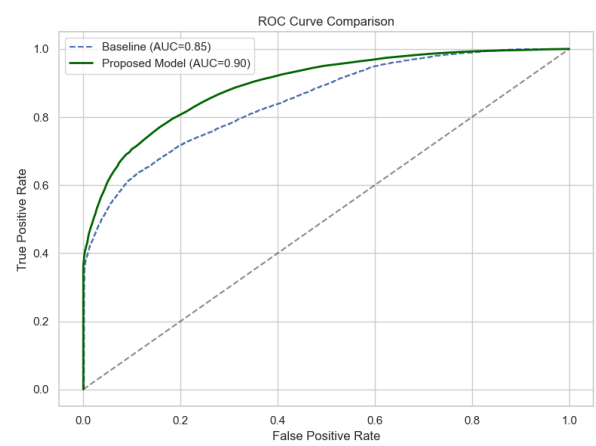

Fig. 6. ROC Curve Comparison. The Random Forest (Green line) outperforms the linear baseline, particularly in maintaining a high True Positive Rate while minimizing False Positives.

C. What Matters Most?

When we asked the model which features were most important (Fig. 8), **Lead Time** and **Deposit Type** came out on top. Interestingly, `total_spending_usd` was also a top

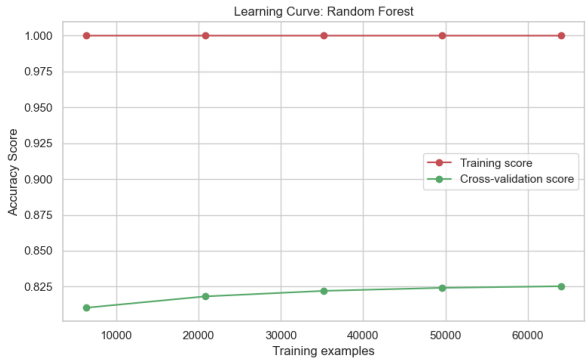

Fig. 7. Learning Curve (Random Forest). The convergence of Training (Red) and Cross-Validation (Green) scores indicates the model generalizes well to unseen data.

factor. This confirms our hypothesis: financial commitment is a huge predictor of whether someone shows up. If a guest has paid a non-refundable deposit or spent a large sum, the psychological cost of cancelling is higher.

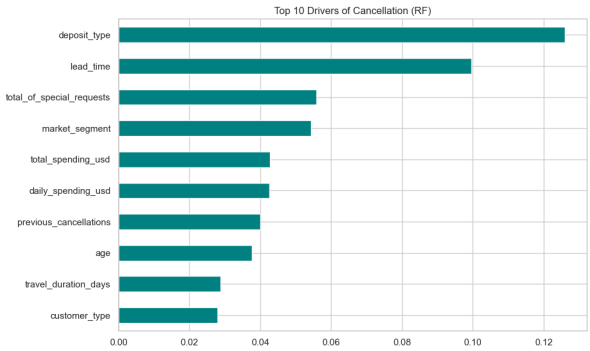

Fig. 8. Feature Importance Plot. 'Lead Time' and 'Deposit Type' are the dominant predictors, followed closely by the spending metrics derived in our EDA.

VII. Discussion and Managerial Implications

These findings aren't just academic; they can be used by real hotel managers in Nepal to improve profitability.

A. Smarter Overbooking

Hotels can plug this model into their systems. The Confusion Matrix (Fig. 9) shows the model is great at identifying confirmed guests. If the model says a certain date has a high risk of cancellations (lots of "Budget Planners"), the manager can safely overbook to ensure the hotel stays full. Conversely, if the model predicts low cancellations (mostly "Luxury" guests), the manager should close bookings early to avoid the expensive mistake of "walking" a high-value guest.

B. Tailored Policies

Instead of treating everyone the same, hotels can treat clusters differently.

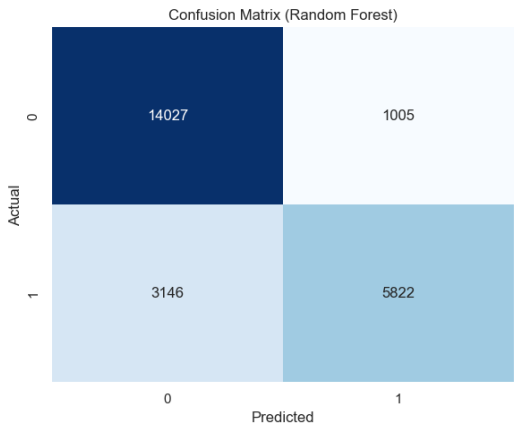

Fig. 9. Confusion Matrix (Random Forest). The model correctly identified the majority of cancellations (True Positives) while minimizing False Negatives.

- **Cluster 0 (Budget Planner):** High risk. Hotels should enforce strict non-refundable policies for bookings made ¿3 months in advance.
- **Cluster 3 (Luxury):** Low risk. Hotels can offer "Free Cancellation" as a perk to attract these high-spending guests, knowing they rarely cancel anyway.

C. Ethical Considerations and Limitations

While effective, automated profiling raises ethical questions. Is it fair to charge one guest more than another based on their profile? We mitigated this by using behavioral data (spending) rather than demographic data (race, gender), but bias is always a risk. Furthermore, the model relies on historical data. If a major external event happens (like a pandemic or natural disaster), historical patterns will break, and the model will need retraining. This "concept drift" is a key limitation of all machine learning models in dynamic environments like tourism.

VIII. Conclusion

This research tackled the expensive problem of cancellations in the Nepalese hospitality sector. By combining K-Means clustering with Random Forest, we moved beyond simple guessing. Our hybrid model achieved **82.70% accuracy** and a robust **AUC of 0.90**, clearly beating the industry-standard logistic regression. The validation metrics prove that profiling guests based on socio-economic factors is a viable way to assess risk. This confirms that revenue leakage is not just bad luck—it is a predictable, manageable problem. In the future, we hope to add real-time weather data to make these predictions even sharper, moving from "static" prediction to "real-time" revenue defense.

References

[1] N. Antonio, A. de Almeida, and L. Nunes, "Hotel Booking Demand Datasets," *Data in Brief*, vol. 22, pp. 41-49, 2019.

[2] L. Morales and J. Wang, "Forecasting cancellation rates for services booking revenue management," *European Journal of Operational Research*, vol. 202, no. 2, pp. 554-562, 2010.

[3] Scikit-learn developers, “User Guide: Clustering and Ensemble Methods,” 2024. [Online].
[4] T. M. Mitchell, *Machine Learning*. New York: McGraw-Hill, 1997.
[5] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.
[6] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
[7] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*. Springer, 2009.

Appendix A
Source Code Availability

To allow others to verify our results, the complete source code, including the data cleaning steps, clustering algorithms, and model configurations, is hosted on GitHub. Researchers can access the repository at:

https://github.com/erwinshrestha17/STW7072CEM-Machine-Learning-Assignment

The repository includes:

- `app.py`: The main script that runs the analysis pipeline.
- `requirements.txt`: A list of Python libraries needed to run the code.
- `README.md`: Instructions on how to set up the environment and reproduce the figures in this paper.